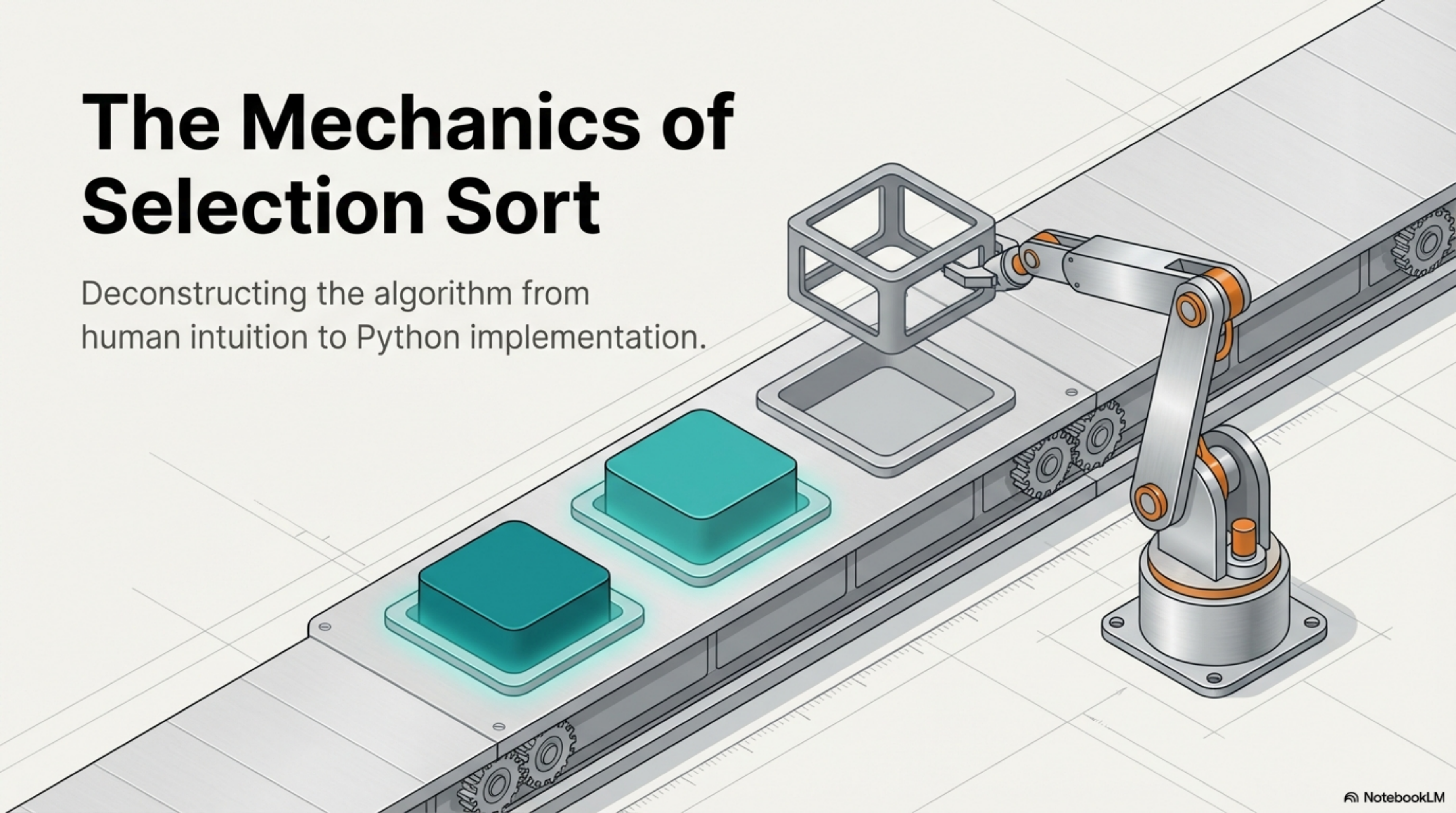


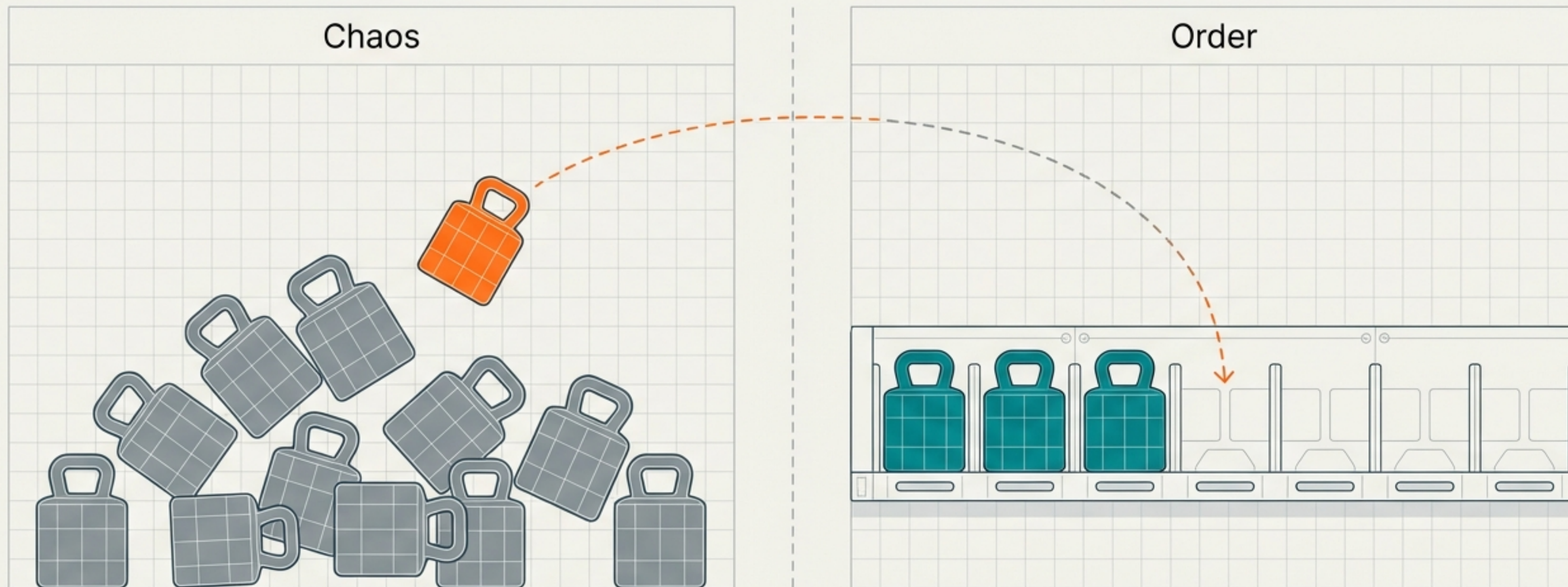
The Mechanics of Selection Sort

Deconstructing the algorithm from human intuition to Python implementation.



The Intuition of Selection

Sorting algorithms mimic human organization. To arrange a chaotic set of items, you **scan the entire** group, **select** the **absolute smallest** item, and lock it into the first position. You then repeat this process for the remaining items.



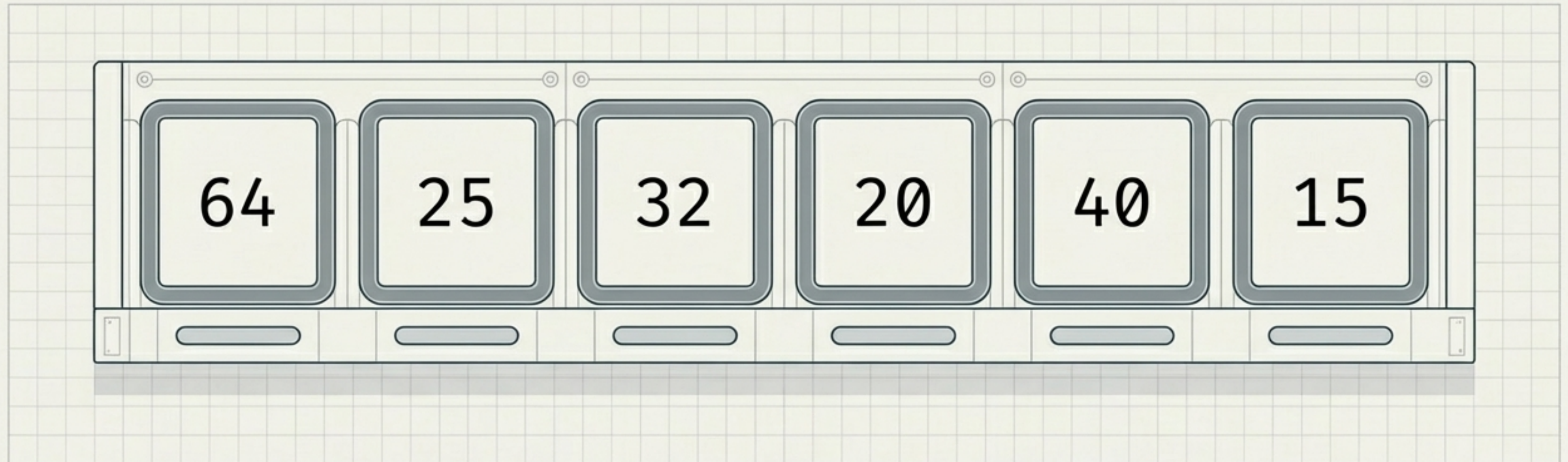
The Sorting Logic Matrix

The Selection Sort algorithm adapts to your target output based on two choices: what you search for, and where you place it.

		Strategy Choice	
		Find the Minimum	Find the Maximum
Target Order	Increasing	Swap with First Unsorted Slot	Swap with Last Unsorted Slot
	Decreasing	Swap with Last Unsorted Slot	Swap with First Unsorted Slot

Initializing the Track

We begin with an unsorted array of six elements. Our goal is strict increasing order by finding the minimum value in each pass.



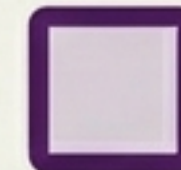
Grey Outline =
Unsorted Region



Solid Teal =
Sorted Region



Bright Orange =
Current Minimum

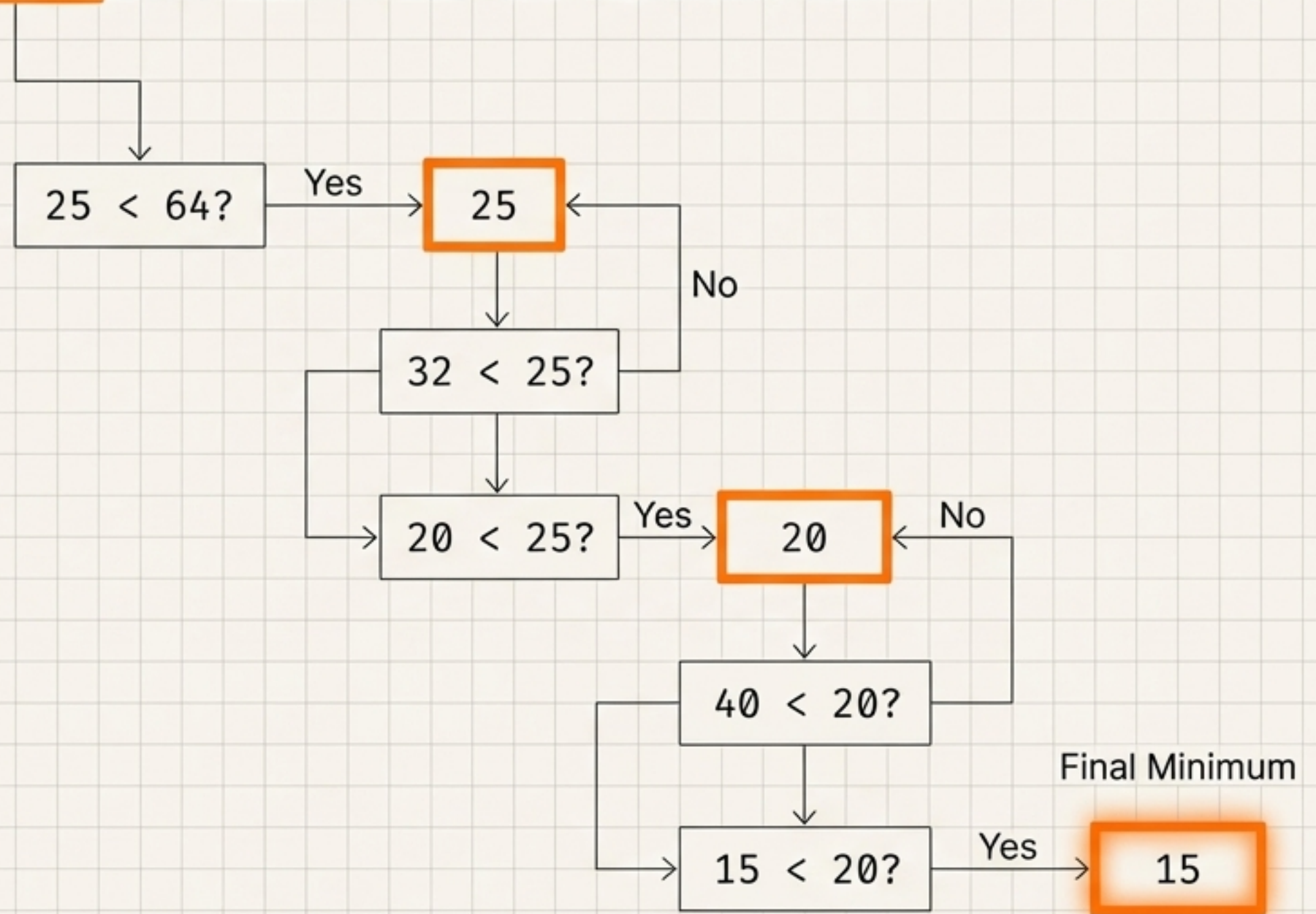


Purple Highlight =
Target / Active Compare

Iteration 1: Scouting the Minimum

We assume the first element is the minimum, then sequentially challenge that assumption against every other element.

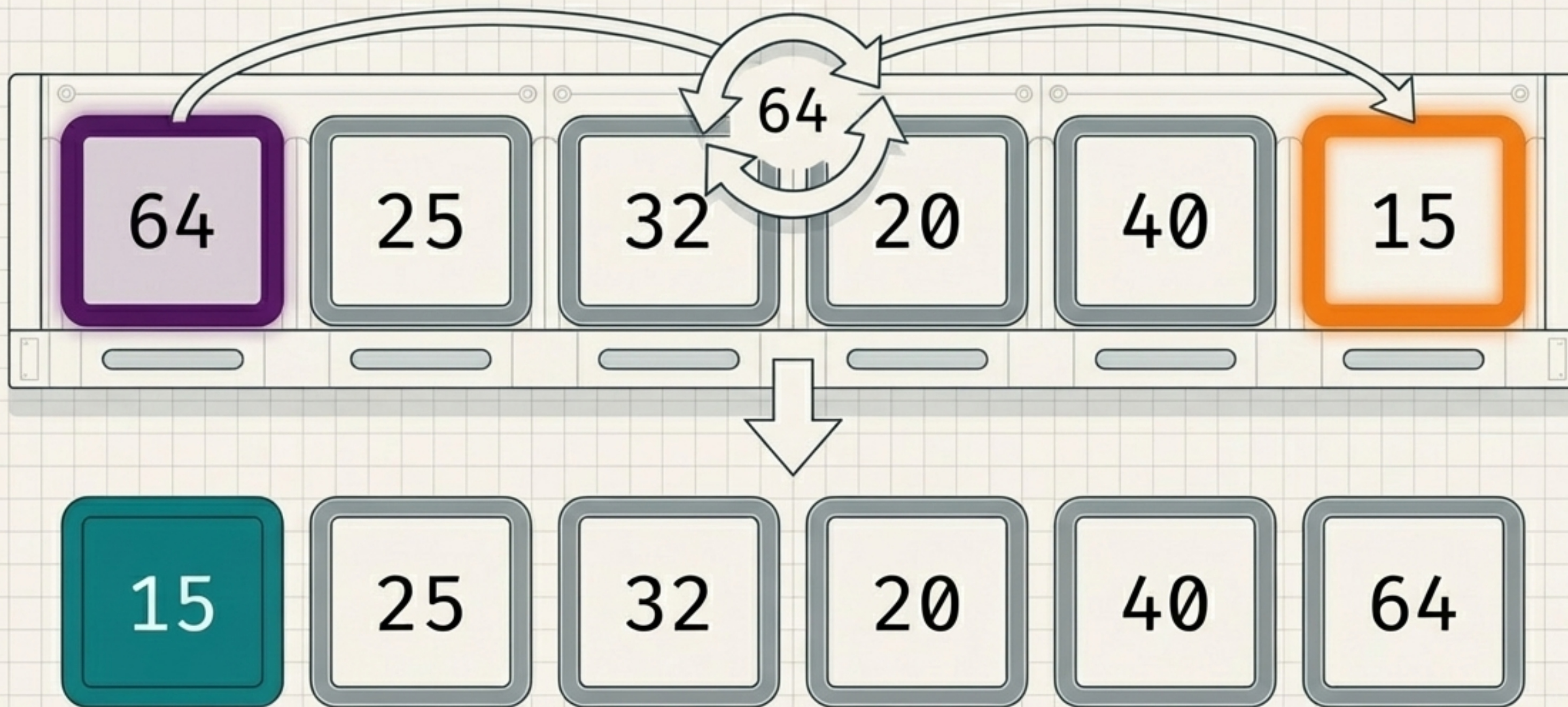
Assume Min



The Pythonic Swap

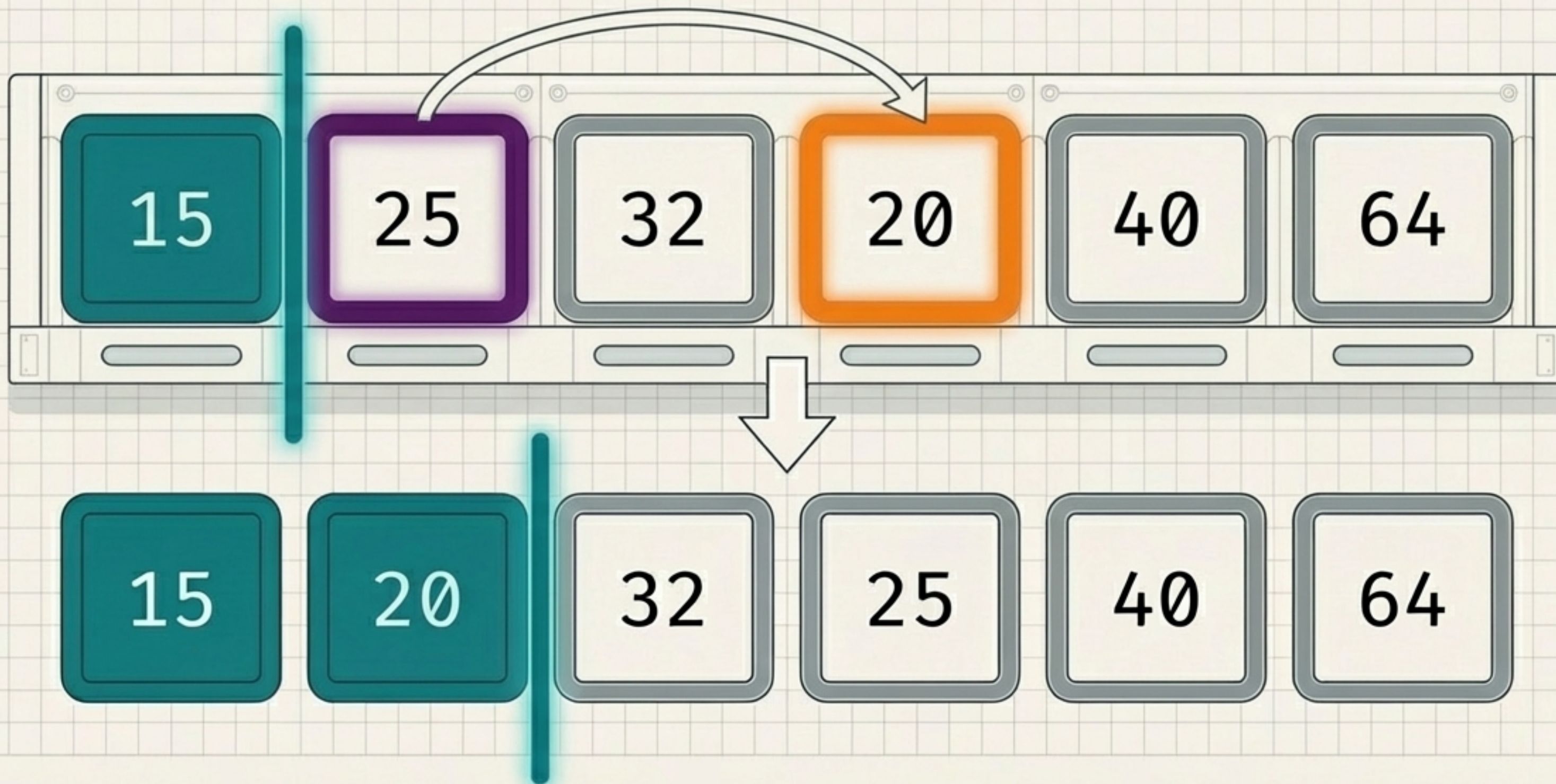
Once the absolute minimum is found, it is swapped directly with the first unsorted position. In Python, this requires no intermediate temporary variables.

```
a, b = b, a
```



The Moving Boundary

With '15' locked into place, a boundary line shifts right. The algorithm now completely ignores the sorted territory, focusing only on finding the minimum in the remaining grey slots.



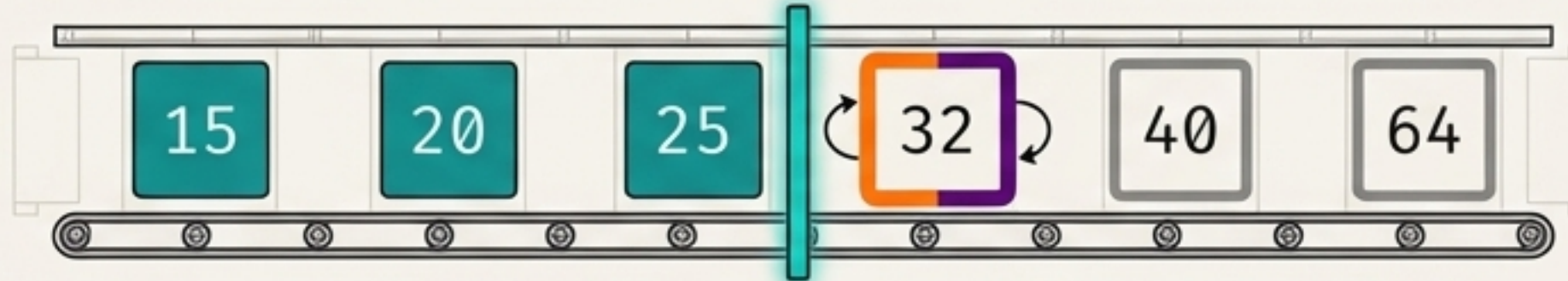
Completing the Assembly Line

The wall continues shifting right until the array is fully sorted. Note that in Iteration 4, '32' is already the minimum of the remaining elements, so it effectively swaps with itself. The final element is automatically sorted.

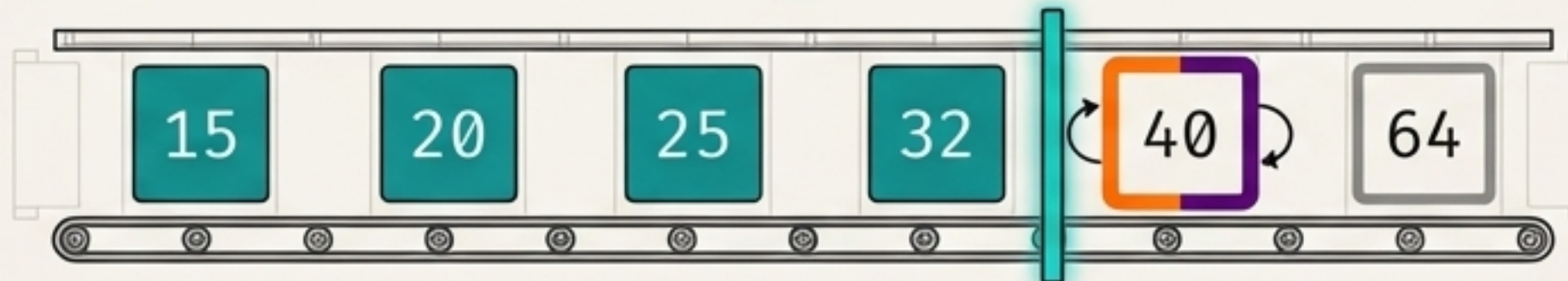
Iteration 3



Iteration 4
(Self-Swap)



Iteration 5



Final Sorted Array



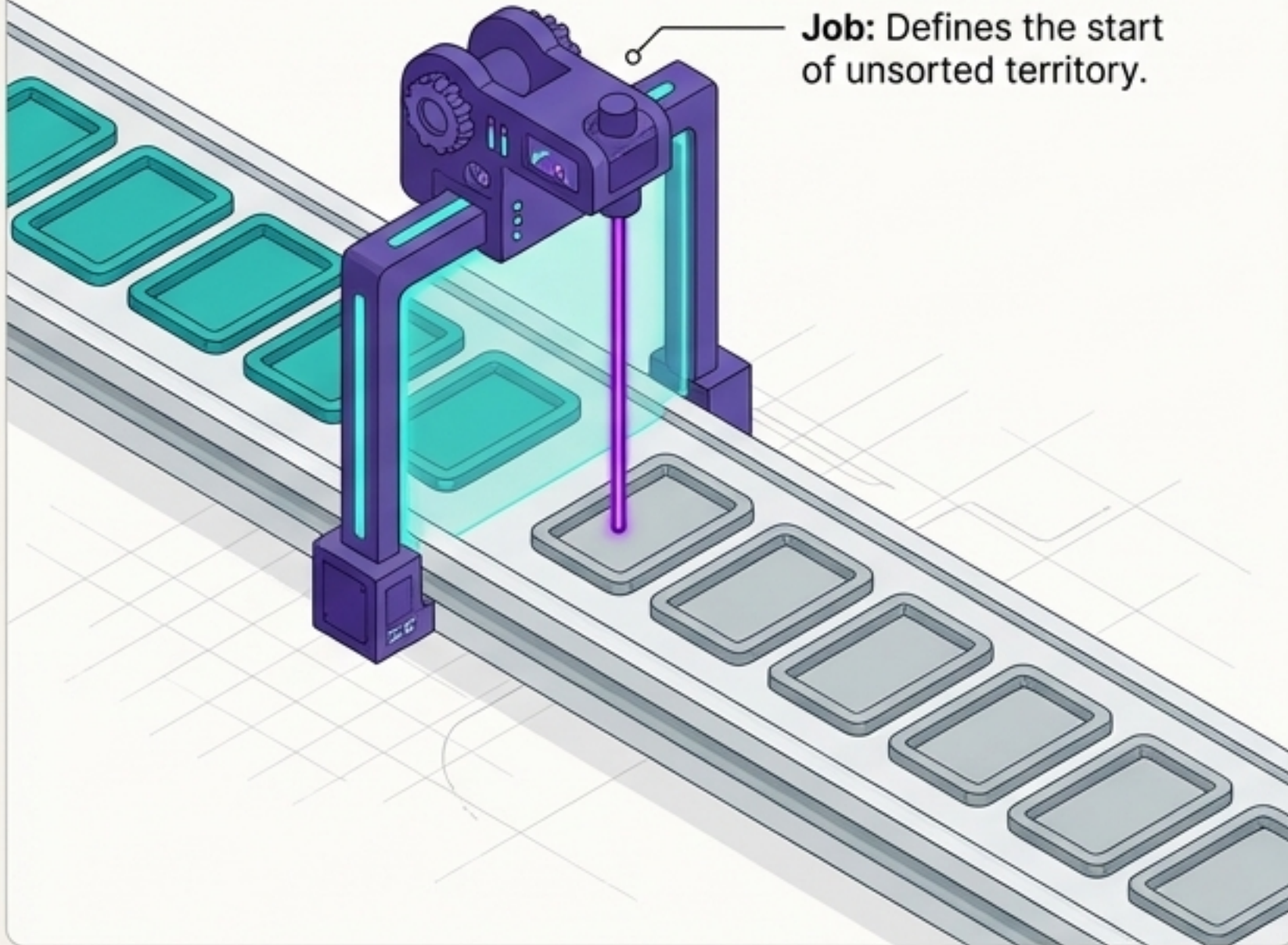
The Dual-Loop Architecture

Selection Sort requires two nested loops working in tandem. The outer loop manages the destination, while the inner loop searches the unknown.

The Manager

Outer Loop (i)

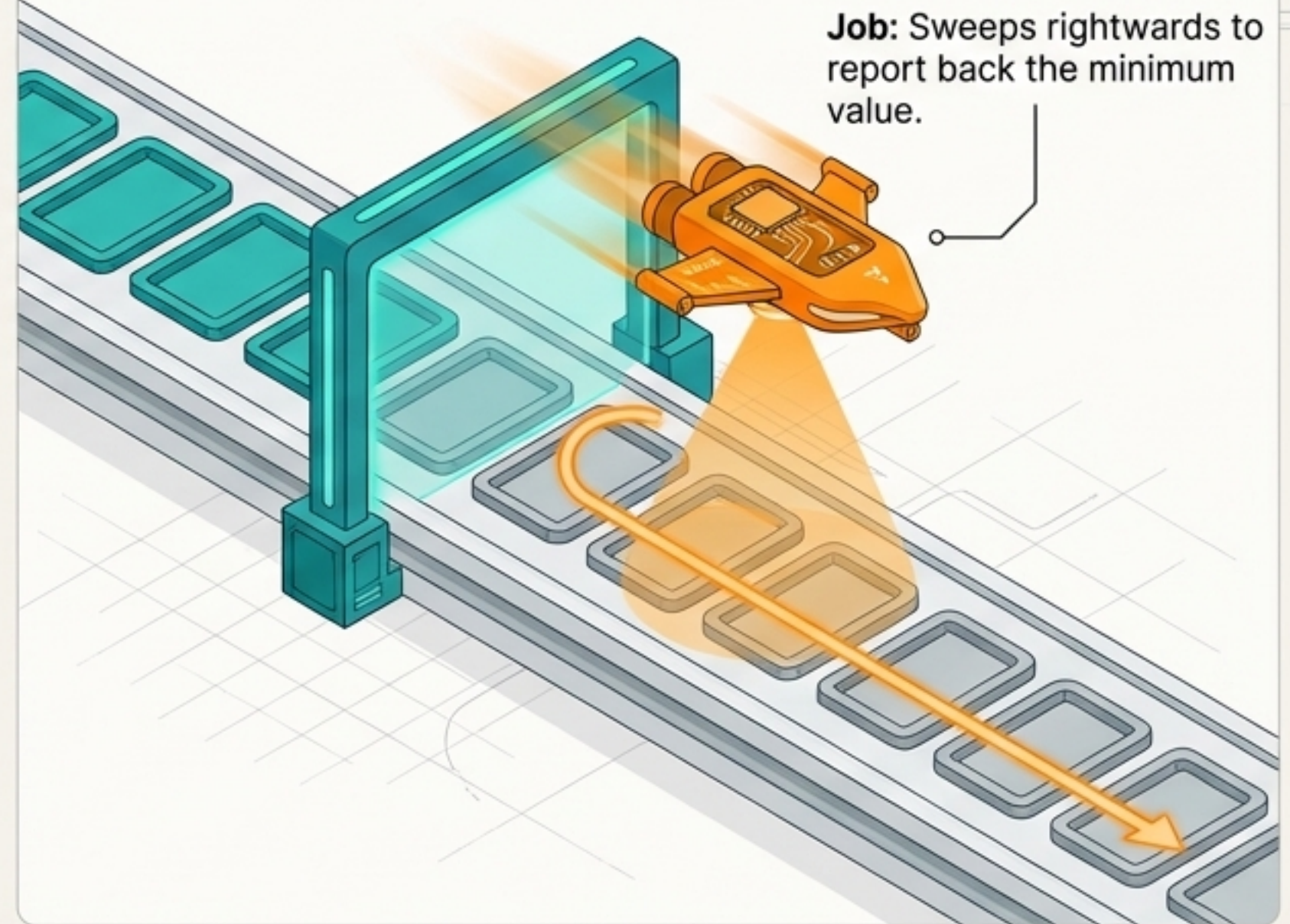
Job: Defines the start of unsorted territory.



The Scout

Inner Loop (j)

Job: Sweeps rightwards to report back the minimum value.



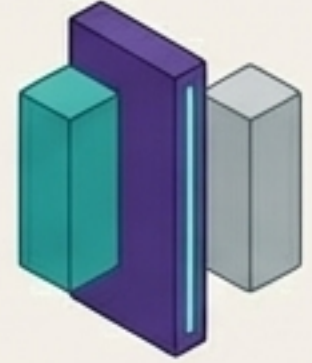
Tracking Locations, Not Values

A critical implementation detail: the tracking variable doesn't store the lowest number itself. It stores the exact address of the lowest number.



Code-to-Visual Anatomy

```
n = len(a)
for i in range(n):
    min_index = i
    for j in range(i, n):
        if a[min_index] > a[j]:
            min_index = j
    a[i], a[min_index] = a[min_index], a[i]
```



The Cost of Nested Loops

Because the outer loop runs n times, and the inner loop runs a shrinking fraction of n times, the total number of operations forms a triangle. The time complexity is bounded by the area of the total square.

